

Joyce Willis

RecipeJoy – Your Personal Recipe Manager



Photo source: generated by AI Image Generator from text at www.fotor.com

Edited by: Joyce Willis

Executive Summary

Problem Statement

Recipes have been endeared since cooking began, passing along secrets from loved ones for generations. Recipes stored using traditional methods like ink and paper are easily misplaced, destroyed, and possibly lost forever. Newer methods of saving document files and favorite links work well if there is an organized filing system and the links never break. A better way to store valued and favorite recipes that can be saved forever and shared for multiple generations is in a single location.

Proposed Solution

RecipeJoy is a software application designed to simplify how users organize, discover, and share their favorite recipes tailored to their unique preferences and dietary needs. RecipeJoy is a centralized hub for users to store and manage their personal recipe collections. With intuitive features and customizable options, RecipeJoy intends to meet these needs by:

- Empowering users to organize recipes based on things such as macros, calories, primary protein types, and user-defined tags.
- Making it easy to find and access favorite recipes that suit their taste preferences and nutritional goals.
- Providing results based on the user-defined search criteria.

Key Features

- Comprehensive recipe management.
- Ability to define and customize foods, recipes, nutritional info, and specific tags.
- Advanced search functionality based on user-defined criteria.

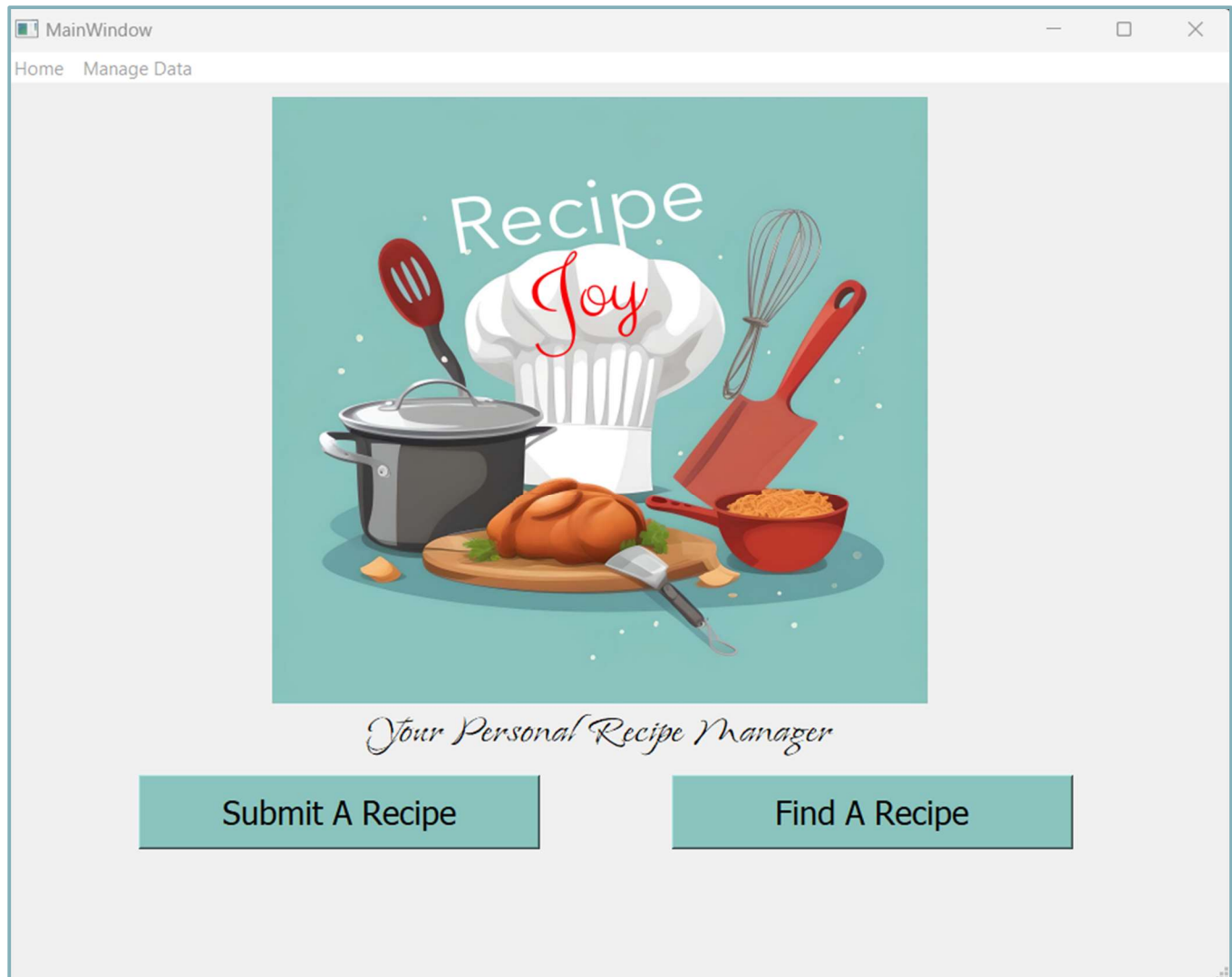
Value

RecipeJoy streamlines organizing, managing, and retrieving specific recipes from your collection of favorites, eliminating the need for cumbersome spreadsheets or paper-based systems and ensuring information is available to pass down for generations.

Users can easily define their meal choices and simplify the meal planning process with the ability to define and store specific information related to foods, recipes, nutritional info, and specific tags by using customizable searches to retrieve recipes within specific parameters. This promotes healthier and more targeted eating habits by making it easy for users to quickly define the choices for their planned meals.

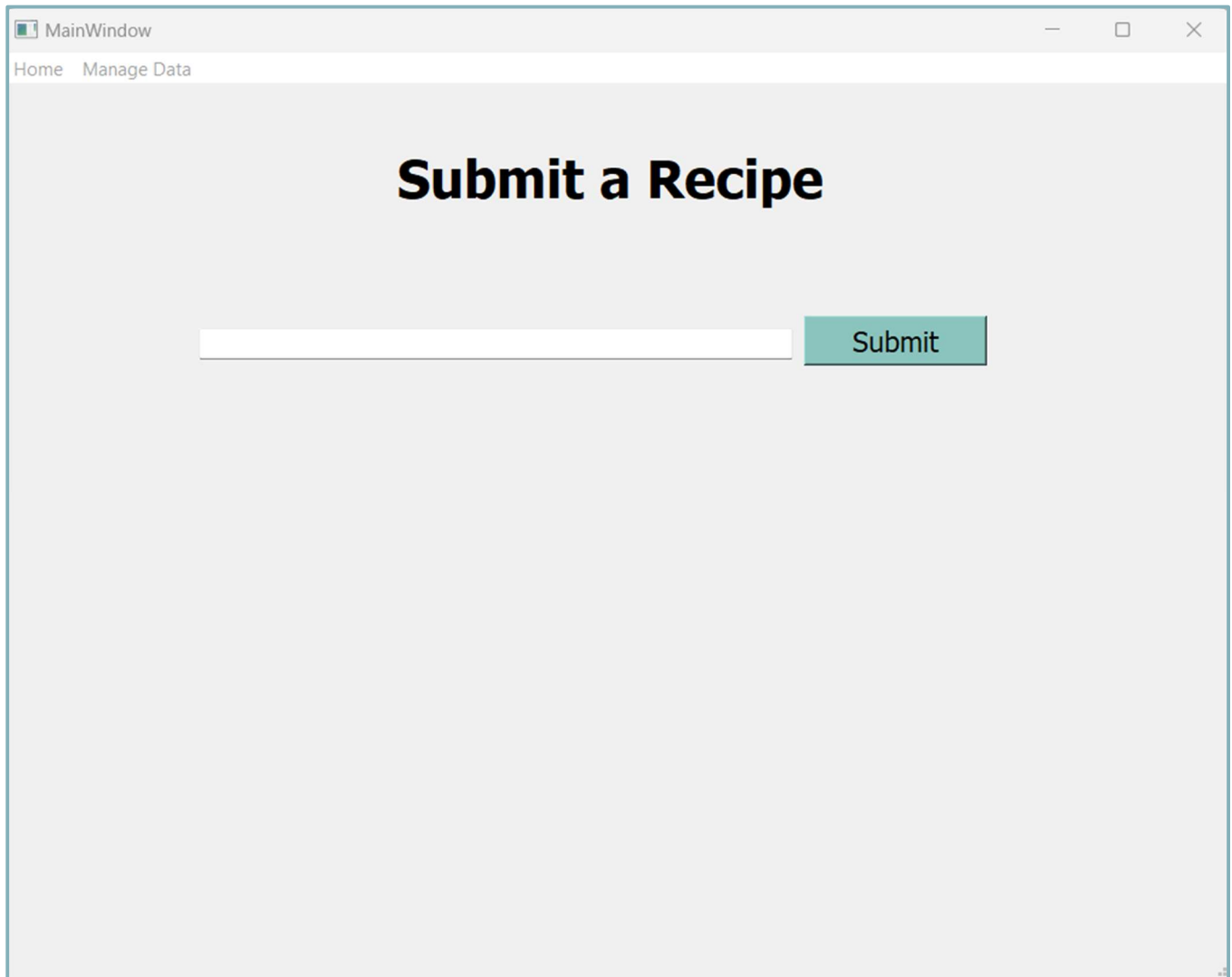
Scenario | Home Page

This is the landing page of the application. From this screen the user can perform actions for submitting a new recipe, finding an existing recipe, and use the menu to navigate to the screens to manage data related to the recipes (custom tags, food items, and protein types).



Scenario | Submit A Recipe

This page allows the user to submit a new recipe name. After the name is confirmed to be new, the “Add Foods” button is enabled, and the user is presented with a message to add foods to the recipe to continue.



The screenshot shows a web application window titled "MainWindow". The window has a light gray background and a white header bar. The header bar contains the text "Home" and "Manage Data" on the left, and standard window control buttons (minimize, maximize, close) on the right. The main content area has a light gray background and features the title "Submit a Recipe" in a large, bold, black font. Below the title is a white text input field with a thin gray border. To the right of the input field is a teal-colored button with the text "Submit" in white. The overall layout is clean and modern.

This screen allows the user to search for foods to be added to the recipe. A remove option is available in cases where a food is selected in error.

MainWindow

HomeManage Data

Add or Edit Foods for Recipe

Find food to add:

Matching Foods:

	Ingredient	Amount	Unit	
1	Olive oil	2	Tablespoon (tbsp)	Add to Recipe
2	Extra light olive oil	1	Cup (c)	Add to Recipe

Recipe Ingredients:

	Food Name	Quantity	Size	Calories	Carbs	Fat	Protein	
1	Extra light olive oil	1	Cup (c)	2481	0	281	0	Remove
2	Egg	1	Whole	74	0	5	6	Remove
3	Mustard powder	1	Teaspoon (tsp)	0	0	0	0	Remove
4	Salt	1	Teaspoon (tsp)	0	0	0	0	Remove
5	Lemon juice	1	Teaspoon (tsp)	0	0	0	0	Remove

MainWindow

HomeManage Data

Add or Edit Steps for Recipe

Enter current step:

After you've added all the oil and the mixture has emulsified, add the lemon juice, blending on low or stirring to incorporate.

Recipe Steps: (Double-click text to edit)

Add Step

	Step	Delete	Move Up	Move Down
1	Place 1/4 cup of the olive oil, the egg, mustard powder, and salt in a blender or food processor and mix thoroughly.	Delete	Move Up	Move Down
2	While the food processor is blending, slowly drizzle in the remaining olive oil.	Delete	Move Up	Move Down

Save and Continue

This page allows the user to enter the optional details of the recipe.

The screenshot shows a web application window titled 'MainWindow' with a navigation bar containing 'Home' and 'Manage Data'. The main content area is titled 'Finish Recipe Info' in a large, bold, black font. Below the title, there are three input fields: 'No of Servings:' with a text box containing '1', 'Add Protein Type:' with a dropdown menu showing 'Select Protein Type', and 'Add Notes:' with a text area containing the text 'The key to the success of this recipe is ensuring all ingredients are used at room temperature.' Below the notes, there is a section titled 'Add Tags:' with a list of checkboxes: 'Brunch', 'Condiment' (checked), 'Dinner', 'Instant Pot', and 'Lunch'. At the bottom center, there is a green button labeled 'Save and Complete'.

MainWindow

Home Manage Data

Finish Recipe Info

No of Servings: 1

Add Protein Type: Select Protein Type

Add Notes:

The key to the success of this recipe is ensuring all ingredients are used at room temperature.

Add Tags:

- ☐ Brunch
- ☒ Condiment
- ☐ Dinner
- ☐ Instant Pot
- ☐ Lunch

Save and Complete

The user is returned to the recipe summary screen with the newly added information.

MainWindow

HomeManage Data

Mayonnaise

Serving Size:
1

Edit Recipe

Ingredients:

	Ingredient	Quantity	Measure
1	Extra light olive oil	1	Cup (c)
2	Egg	1	Whole
3	Mustard powder	1	Teaspoon (tsp)

Steps:

Steps

1. Place 1/4 cup of the olive oil, the egg, mustard powder, and salt in a blender or food processor and mix thoroughly.

2. While the food processor is blending, slowly drizzle in the remaining olive oil.

3. After you've added all the oil and the mixture has emulsified, add the lemon juice, blending on low or stirring to incorporate.

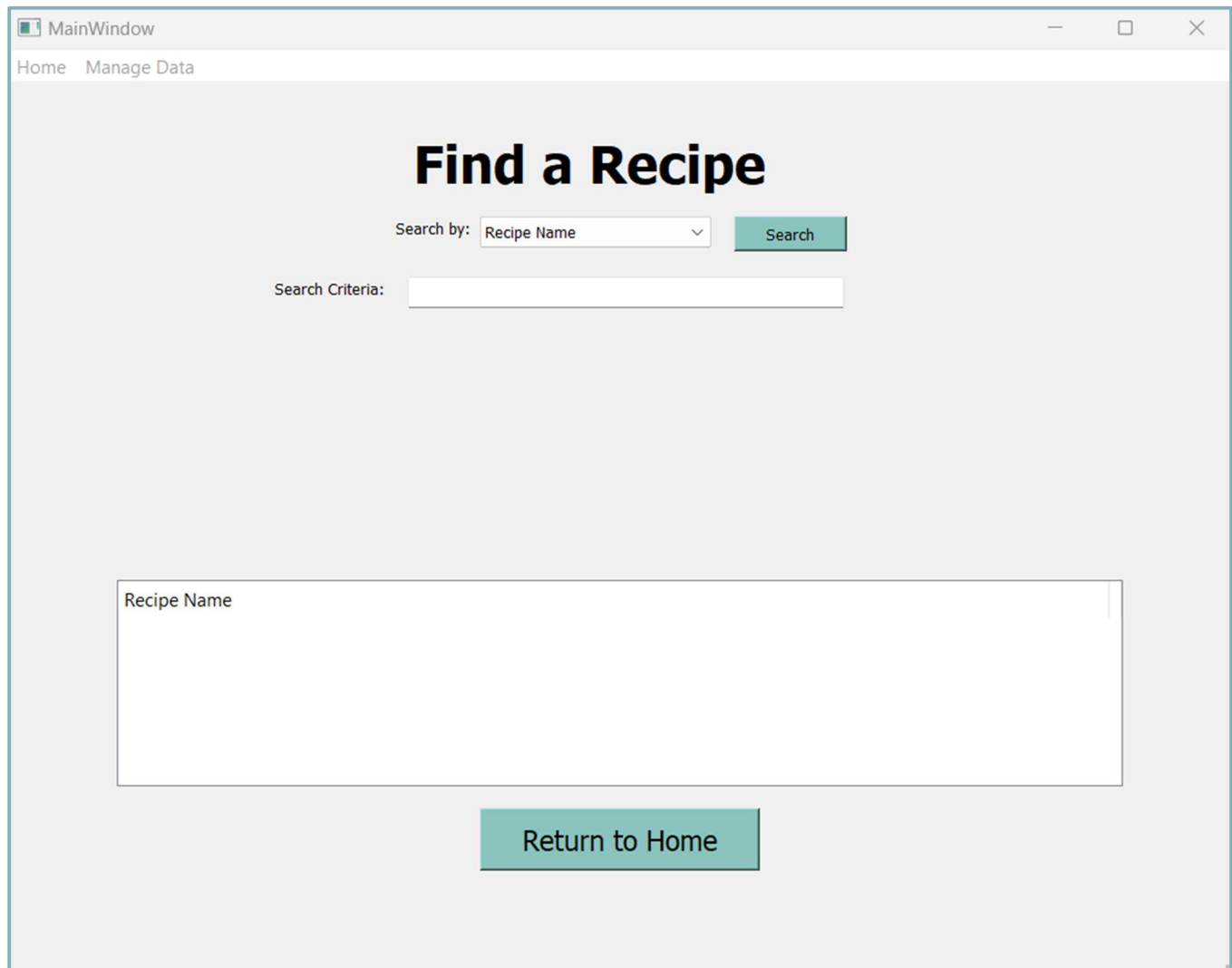
Additional details:

NotesNutritionTags

The key to the success of this recipe is ensuring all ingredients are used at room temperature.

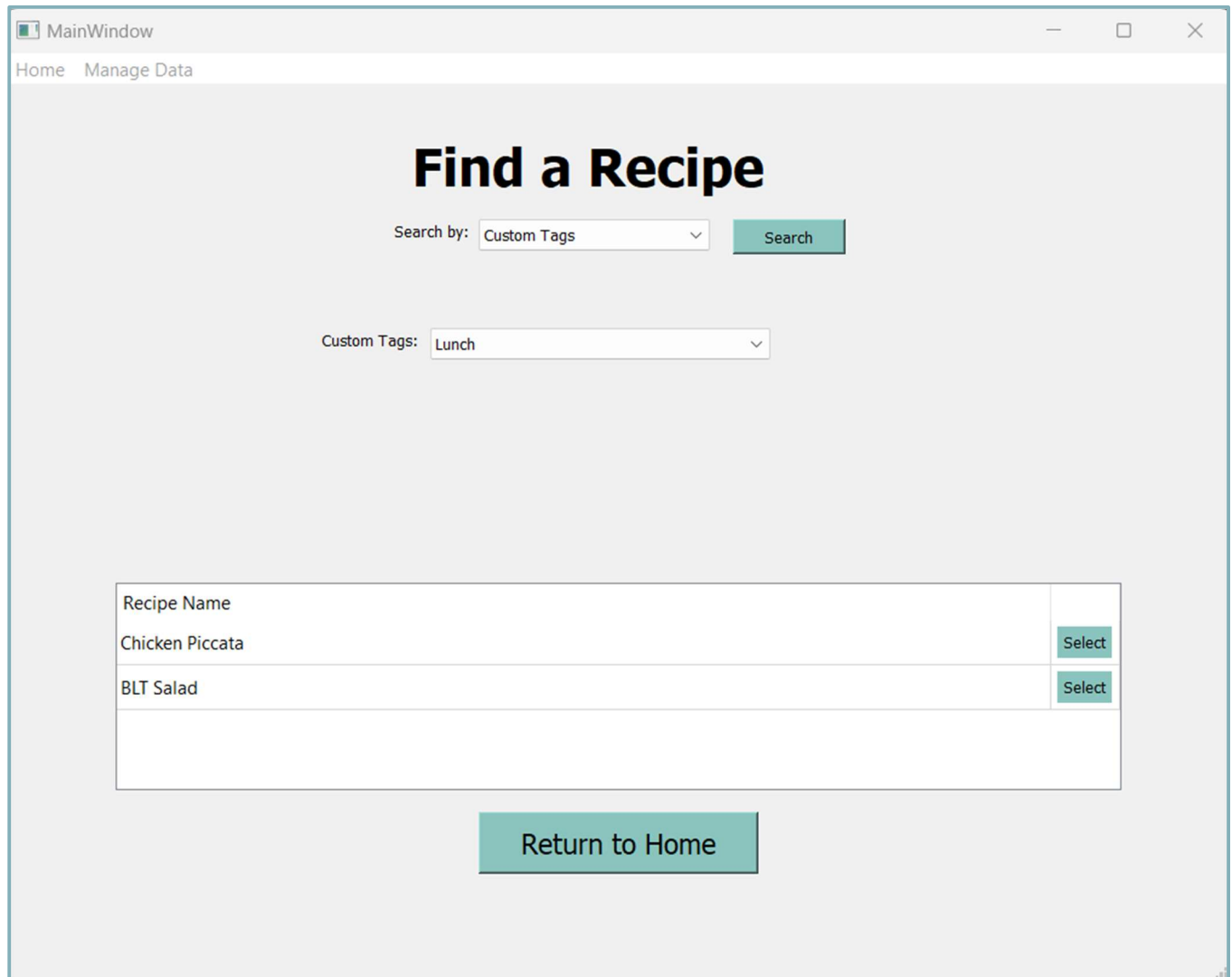
Scenario | Find A Recipe

This page presents the user with a drop-down of options for searching recipes. The defined options are Calorie Range, Custom Tags, Macros, Protein Type, and Recipe Name. The UI search fields are presented to the user depending on the type of search they select.



The screenshot displays a web application window titled "MainWindow". The interface features a navigation bar with "Home" and "Manage Data" links. The main heading is "Find a Recipe". Below this, there is a "Search by:" label followed by a dropdown menu currently set to "Recipe Name" and a teal "Search" button. A "Search Criteria:" label is positioned above a large, empty white text input field. At the bottom center, there is a teal button labeled "Return to Home".

The search results are presented to the user where they have an option to select the desired recipe.



MainWindow

Home Manage Data

Find a Recipe

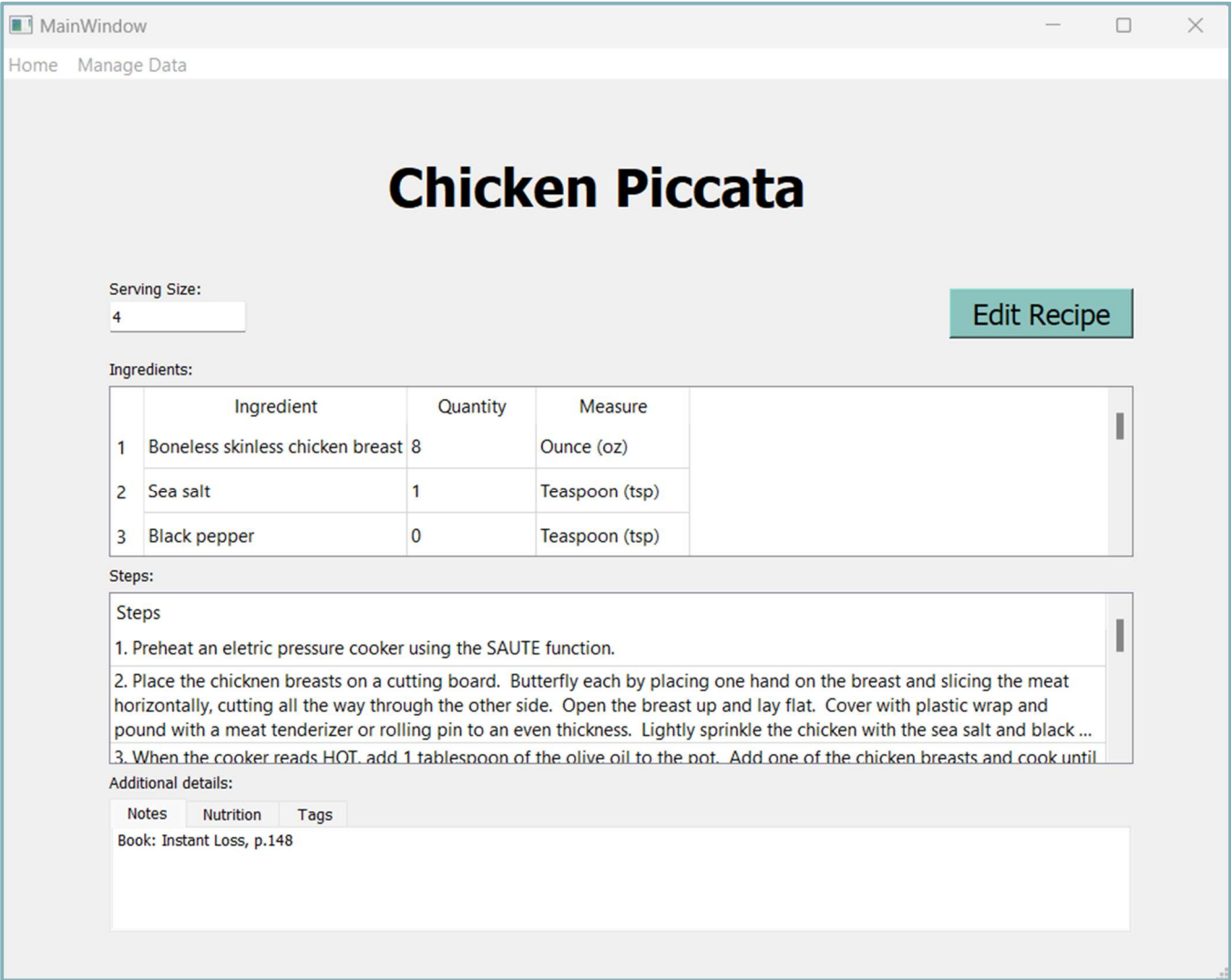
Search by: Custom Tags

Custom Tags: Lunch

Recipe Name	
Chicken Piccata	Select
BLT Salad	Select

Scenario | View Recipe

This screen presents the user with the recipe details.



The table at the bottom shows details such as notes (in image above), nutrition, and tags (shown below).

Additional details:

Notes

Nutrition

Tags

	Calories	Carbs (g)	Fat (g)	Protein (g)
1	668	0	54.00	46.00

Additional details:

Notes

Nutrition

Tags

Lunch

Dinner

Instant Pot

Scenario | Edit Recipe

The edit recipe option navigates the user back through the same process and steps of adding a recipe where foods, steps, and recipe details can be added, deleted, and updated as needed.

MainWindow

HomeManage Data

Add or Edit Foods for Recipe

Find food to add:

Search

Matching Foods:

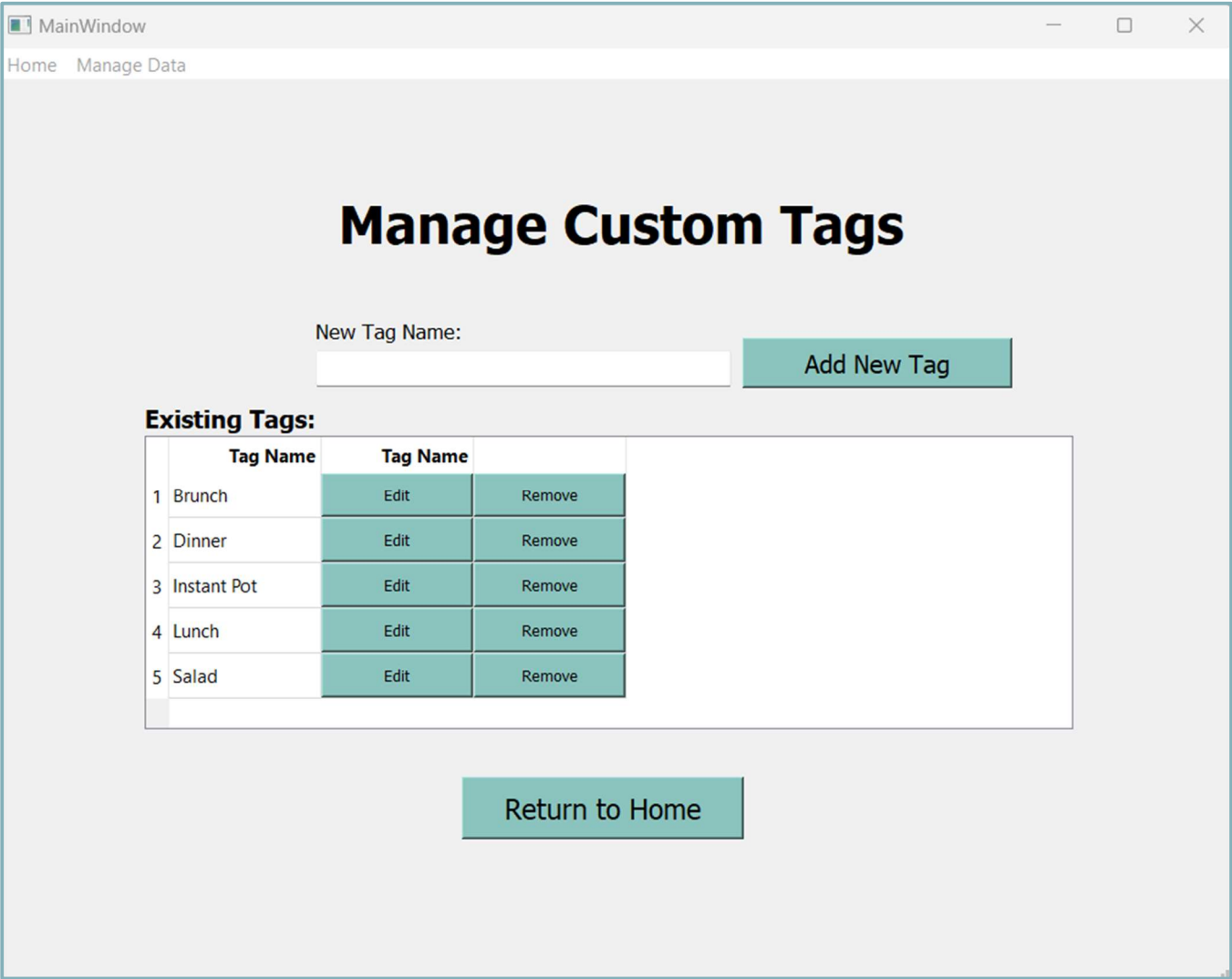
Recipe Ingredients:

	Food Name	Quantity	Size	Calories	Carbs	Fat	Protein	8
1	Boneless skinles...	8	Ounce (oz)	220	0	3	46	Remove
2	Sea salt	1	Teaspoon ...	0	0	0	0	Remove
3	Black pepper	0	Teaspoon ...	0	0	0	0	Remove
4	Olive oil	2	Tablespoo...	238	0	27	0	Remove

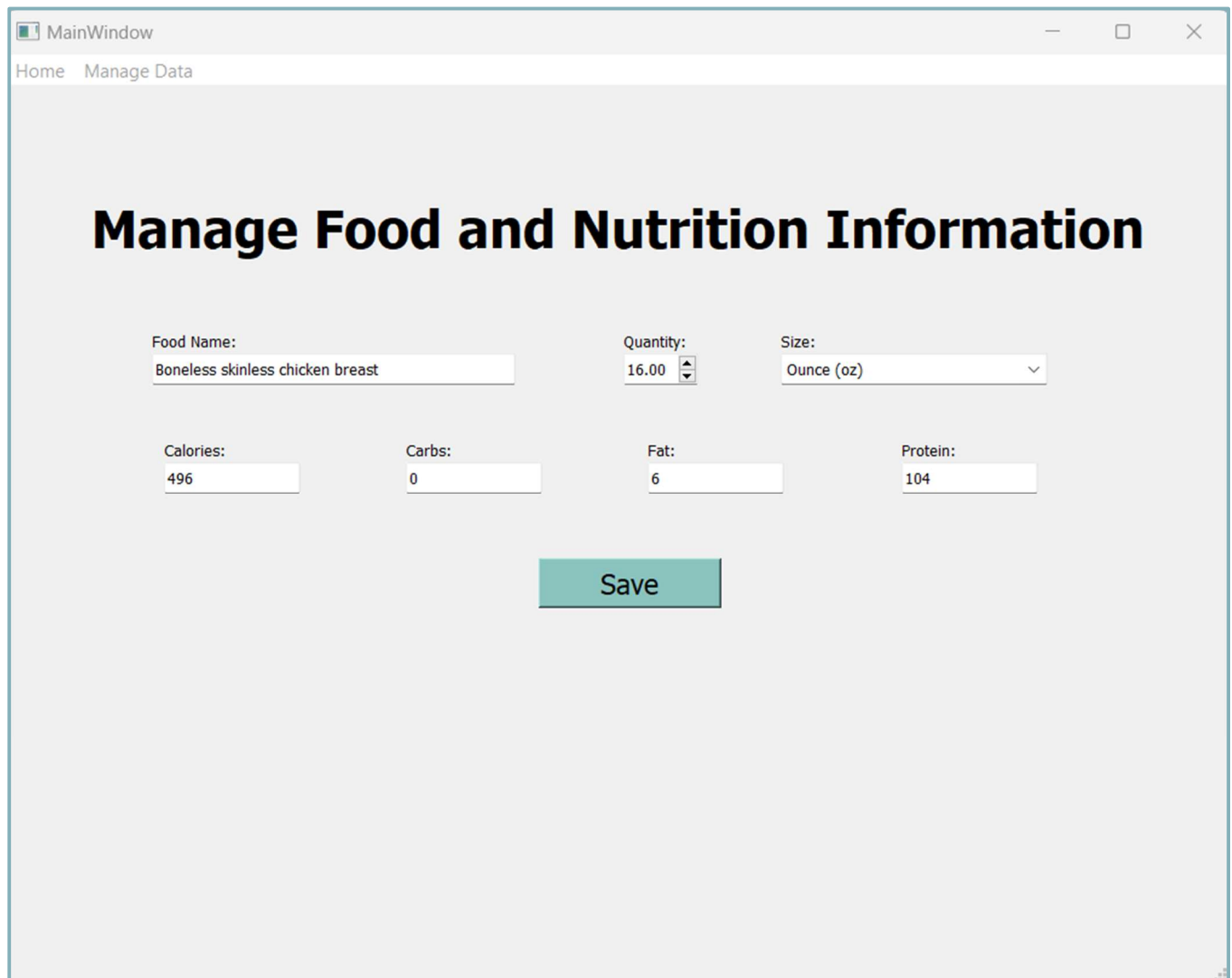
Save and Continue

Scenario | Manage Custom Tags

This screen provides the user with the ability to manage custom tags within the system.



Editing a food presents the user with the information to be revised.



The screenshot shows a web application window titled "MainWindow" with a navigation bar containing "Home" and "Manage Data". The main content area is titled "Manage Food and Nutrition Information". The form contains the following fields:

Food Name:	Quantity:	Size:
Boneless skinless chicken breast	16.00	Ounce (oz)

Calories:	Carbs:	Fat:	Protein:
496	0	6	104

Below the form is a green "Save" button.

Scenario | Manage Protein Types

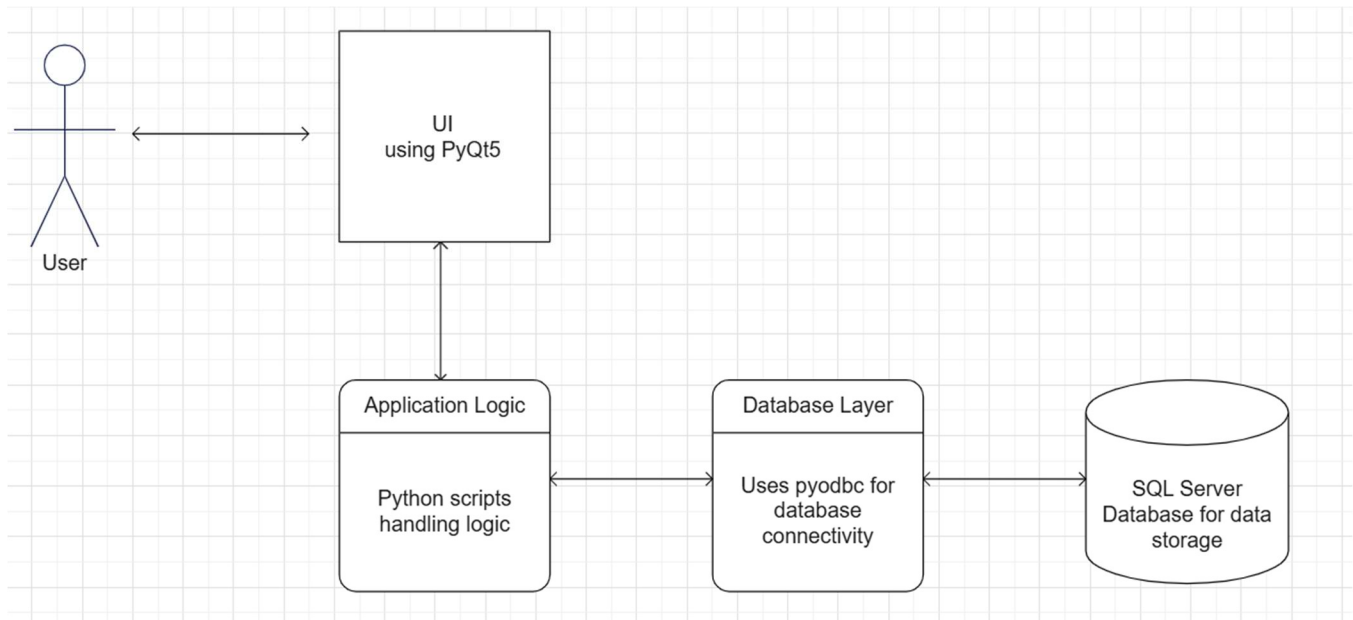
This screen provides the user with the ability to manage protein types within the system.

The screenshot shows a web application window titled 'MainWindow'. It has a navigation bar with 'Home' and 'Manage Data' links. The main content area is titled 'Manage Protein Types'. Below the title, there is a form for adding a new protein type, consisting of a text input field labeled 'New Protein Type:' and a teal 'Add New Protein Type' button. Below this is a table with four rows of protein types: Beef, Chicken, Lamb, and Pork. Each row has an 'Edit' button and a 'Remove' button. At the bottom of the page, there is a teal 'Return to Home' button.

	Protein Type		
1	Beef	Edit	Remove
2	Chicken	Edit	Remove
3	Lamb	Edit	Remove
4	Pork	Edit	Remove

System Architecture

RecipeJoy is a desktop application built using PyQt5 for the user interface and SQL Server for data storage. The application uses a client-server architecture where the PyQt5 application acts as the client, and the SQL Server database serves as the backend.



Source Code Structure

The RecipeJoy project is organized into modular components, each focusing on specific functionalities. The following is a summary of the source code directories and their contents:

Code Directory	
Directory	Usage
RecipeJoy	Root directory of the project.
Assets	Directory for storing project assets.
--logo.png	Project logo image file.
UI_Files	Directory containing UI-related files.
--mainwindow.ui	Qt Designer UI file for the main window.
--ui_mainwindow.py	Python file generated from mainwindow.ui.
custom_dialogs.py	Handles custom dialog boxes for the application.
customTags_operations.py	Handles operations related to custom tags features.
database.py	Handles database operations and connections.
food_operations.py	Handles operations related to foods and nutrition features.
main.py	Main entry point of the application.
proteins_operations.py	Handles operations related to protein features.
recipes_operations.py	Handles operations for recipe management.
ui_operations.py	Handles general UI-related operations.

Highlighted rows indicate directories containing source code.

Code Architecture

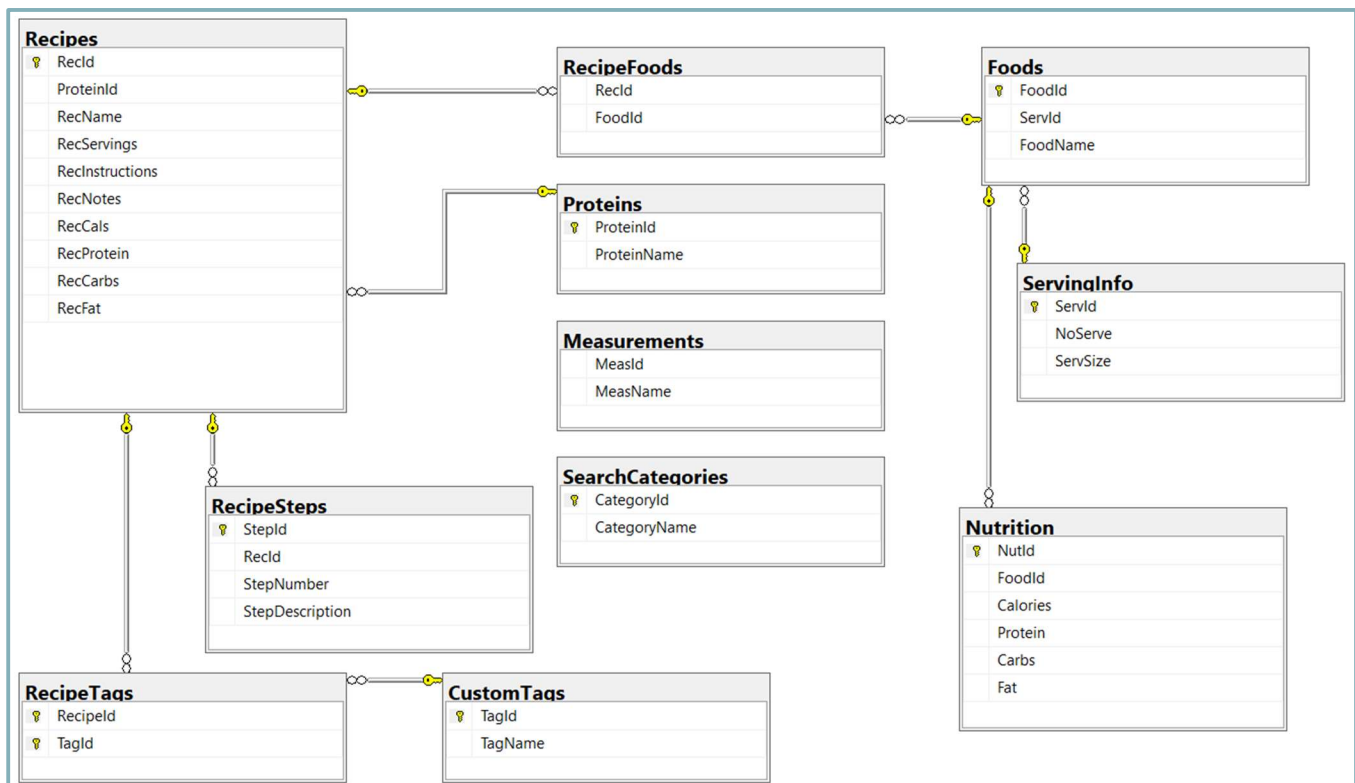
The system is organized into three primary layers: the User Interface (UI) Layer, the Business Logic Layer, and the Database Layer.

- The User Interface (UI) Layer is built using the Qt framework, allowing for a rich graphical interface. This layer uses widgets and includes all of the user-facing components, such as forms, tables, and input fields. Users can navigate between different pages, such as those for editing custom tags, managing food and nutritional information, submitting recipes, and searching for recipes.
- The Business Logic Layer is the backend logic, containing core functionality, implemented in Python. This includes things such as database interactions, data validation, and business rules. This layer handles the processing of user inputs, querying the database, updating UI elements based on database changes, and managing the flow of data between the UI and database.

- The Database Layer serves as the central data store, where all the application's persistent data is maintained. This relational model consists of multiple tables that store information related to recipes, ingredients, nutritional data, serving sizes, and custom tags.

Database or Data Store

The underlying database for RecipeJoy is a relational database designed to manage various aspects of recipes, ingredients, and nutritional information. Each table in the schema plays a specific role in organizing and relating data, ensuring that the application can efficiently handle user requests and maintain data integrity. This relational design allows for complex queries to be performed efficiently, supporting features like multi-criteria searches, dynamic updates to recipes, and accurate nutritional tracking. The system's code interacts with this database to present users with the data they need, while also ensuring that any updates are reflected across the UI in real-time.



Database Schema

- **Recipes Table** – stores core information about each recipe, including name, servings, instructions, and nutritional data.
- **Foods and Nutrition Tables** – the **Foods** table stores individual food items, while the **Nutrition** table keeps track of their nutritional information, such as calories, protein, carbs, and fat.
- **RecipeFoods Table** – This table contains a join to link recipes to their associated foods.
- **ServinInfo Table** – This table contains information about serving sizes, linking foods to provide context for the nutritional data.
- **Proteins, Measurements, and SearchCategories Tables** – these tables store additional information for recipes.
- **RecipeSteps and RecipeTags Tables** – the **RecipeSteps** manages the sequential steps for preparing a recipe, and the **RecipeTags** table links recipes with custom tags for easier searching and categorization.

Programming Language | Python

RecipeJoy is developed using the Python programming language.

Key Components and References

- PyQt5 is used for the main GUI framework, provides Python bindings for Qt5, and is used for creating the desktop application interface.
- Pyodbc is the Python module for ODBC database access and is used for connecting to and interacting with SQL Server database.
- SQL Server is the backend database system which stores all recipe data, food items, custom tags, and related information.

External Dependencies

- Qt5 DLLs are used by PyQt5 for GUI rendering and functionality and are typically installed automatically with PyQt5.

Development Tools:

- Qt Designer is used for designing the UI layouts and generates the files that are converted to Python code.

Project Classes

Classes within the project are used to abstract re-usable pieces of code. Classes are also used to group related values, known as properties. The project utilizes these classes:

Main Window | `ui_operations.py`

The `MainWindow` class is the core of the application, handling UI operations, database interactions, and overall application flow.

UI Setup | `ui_mainwindow.py`

The `Ui_MainWindow` class sets up the user interface, defining all UI elements and their properties.

Database Manager | `database.py`

The `DatabaseManager` class handles database connections and provides methods for connecting to and closing the database.

Custom Tag Operations | `customTags_operations.py`

The `CustomTagOperations` class provides methods for managing custom tags, including adding, updating, deleting, and retrieving tags.

Food | `food_operations.py`

The `Food` class represents a food item with properties like name, serving size, and nutritional information.

Food Operations | `food_operations.py`

The `FoodOperations` class contains static methods for managing food items in the database, including adding, updating, deleting, and searching for foods.

Protein Operations | `proteins_operations.py`

The `ProteinOperations` class provides static methods for managing protein types in the database.

Recipe Operations | `recipes_operations.py`

The `RecipeOperations` class contains static methods for managing recipes, including adding, updating, deleting, and searching for recipes.

Tag Removal Dialog | `custom_dialogs.py`

The `TagRemovalDialog` class creates a custom dialog for confirming tag removal and handling associated recipes.

Food Removal Dialog | `custom_dialogs.py`

The `FoodRemovalDialog` class creates a custom dialog for confirming food item removal and handling associated recipes.

Protein Removal Dialog | `custom_dialogs.py`

The `ProteinRemovalDialog` class creates a custom dialog for confirming protein type removal and handling associated recipes.

Custom Button | `ui_operations.py`

The `CustomButton` class extends `QPushButton` to include row information for table operations.

Project Modules

The modules used in RecipeJoy separate different functionalities and improve code organization. Here is a list of the project's modules:

Main Application Entry | `main.py`

This module serves as the entry point for the application. It initializes the main window and starts the application loop.

User Interface Operations | `ui_operations.py`

This module contains the `MainWindow` class and handles all user interface operations, event handling, and application logic.

User Interface Definition | `ui_mainwindow.py`

This module defines the user interface layout and widgets. It's likely generated from a Qt Designer file.

Database Management | `database.py`

This module handles database connections and provides a `DatabaseManager` class for interacting with the SQL Server database.

Custom Tags Operations | `customTags_operations.py`

This module provides functions and classes for managing custom tags, including adding, updating, deleting, and retrieving tags.

Food Operations | `food_operations.py`

This module contains classes and functions for managing food items, including adding, updating, deleting, and searching for foods.

Protein Operations | `proteins_operations.py`

This module provides functions for managing protein types in the database, including adding, updating, and deleting protein types.

Recipe Operations | `recipes_operations.py`

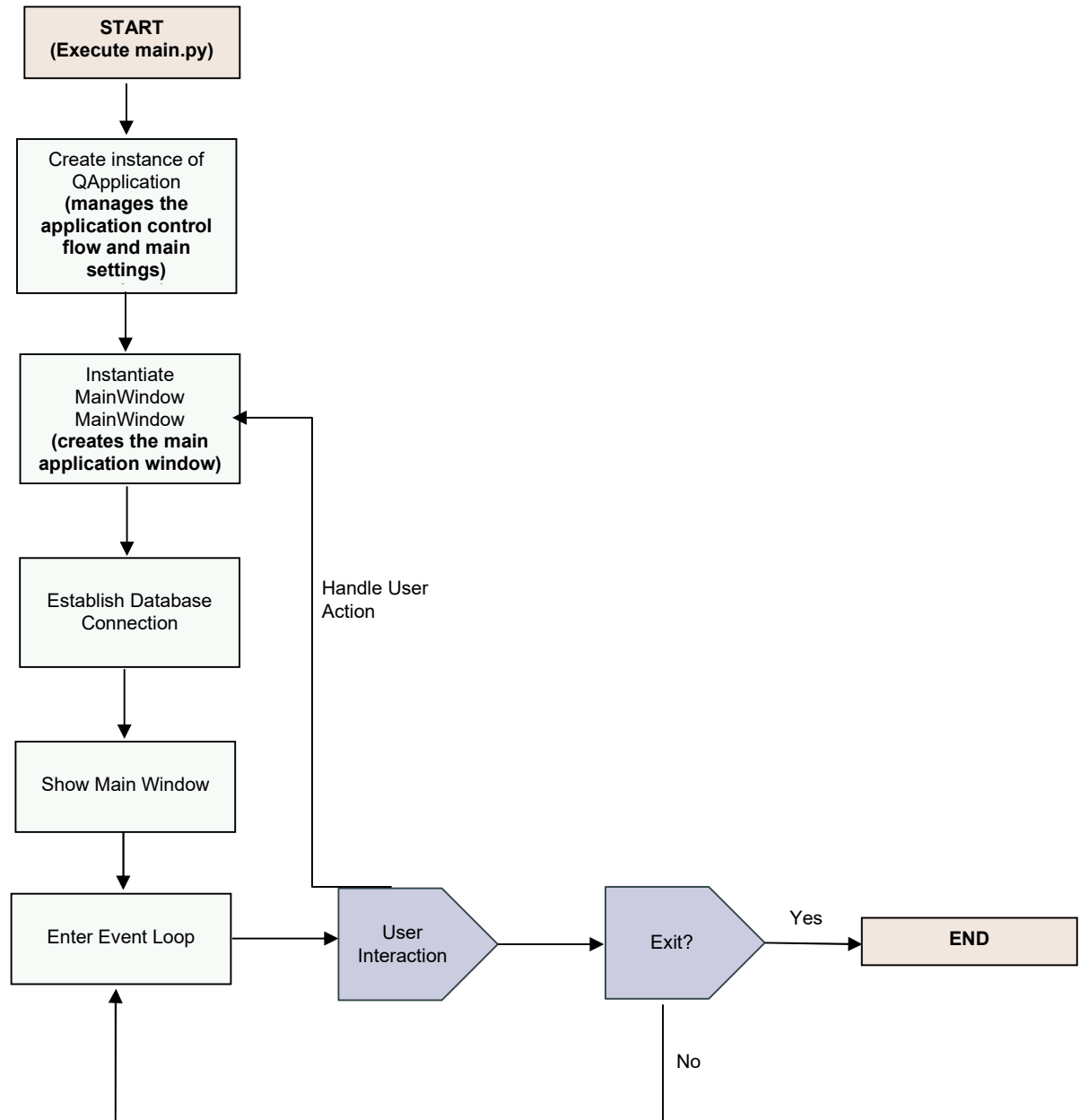
This module contains functions for managing recipes, including adding, updating, deleting, and searching for recipes.

Custom Dialogs | `custom_dialogs.py`

This module defines custom dialog classes for various user interactions, such as confirming deletions and handling associated items.

Program Start and End Flow

Below is a diagram that describes the program flow for RecipeJoy.



Summary

RecipeJoy is a comprehensive desktop application designed for efficient recipe management and meal planning. Developed in Python using the PyQt5 framework, it provides a user-friendly interface for creating, editing, and searching recipes, as well as managing food items and custom tags.

Key features of the system include:

- Recipe creation and editing with ingredient management
- Advanced recipe search functionality
- Custom tag management for recipe categorization
- Food and nutrition information database
- Protein type categorization for recipes

The application is built on a modular architecture, comprising the following key components:

- A main window interface (MainWindow class) handling user interactions
- Database management using pyodbc for SQL Server connectivity
- Specialized modules for recipe, food, tag, and protein operations

RecipeJoy utilizes a SQL Server database for data persistence, ensuring robust storage and retrieval of recipe and nutrition information. The system's flow begins with the main.py entry point, initializing the application and main window, and then enters an event loop to handle user interactions.

GitHub Link

<https://github.com/joywill24/RecipeJoy>

Program & Validation Walkthrough Link (Too large for GitHub upload)

[https://drive.google.com/file/d/1YQVn0grwlvXQ8fAFG0ogguRkwwaF-pNm/view?usp=drive link](https://drive.google.com/file/d/1YQVn0grwlvXQ8fAFG0ogguRkwwaF-pNm/view?usp=drive_link)

Database Walkthrough

[RecipeJoy/DatabaseReview.mp4 at main · joywill24/RecipeJoy \(github.com\)](#)

Reusable Code

[RecipeJoy/ReusableCode.mp4 at main · joywill24/RecipeJoy \(github.com\)](#)

APPENDIX B (BUILD AND RELEASE PROCESS)

The following outlines the build and release process required to implement an update for RecipeJoy:

- Development and Testing
 - Implement new features or bug fixes in the development environment.
 - Run unit tests to ensure new changes don't break existing functionality.
 - Perform thorough manual testing of the application.
- Version Control
 - Commit all changes to the version control system, GitHub.
 - Create a new version tag or branch for the release.
- Database Updates
 - If the update includes database schema changes:
 - Create SQL migration scripts to update the existing database structure.
 - Test the migration scripts in a staging environment.
- Update Version Information
 - Update the version number of the application.
 - Update any relevant documentation with new version information.
- Testing the Build
 - Install the packaged application on a clean test system.
 - Verify that all features work correctly in the packaged version.
 - Test the update process, including any database migrations.
- Documentation
 - Update the user manual with any new features or changes.
 - Create release notes detailing the changes in this version.
 - Update GitHub README file.
- Release Announcement
 - Inform users about the new update through appropriate channels.
- Post-Release Monitoring
 - Monitor user feedback and bug reports after the release.
 - Be prepared to quickly address any critical issues that arise.
- Archiving
 - Archive the build artifacts and exact source code used for this release.

APPENDIX C (CLIENT INSTALLATION INSTRUCTIONS)

The following provides detailed instructions for installing and setting up RecipeJoy on a client machine.

Prerequisites

Before installing RecipeJoy, ensure your system meets the following requirements:

1. Python 3.5 or later is installed
2. PyQt5 library
3. pyodbc library
4. SQL Server Express 2019 or later

Installation Steps

1. **Install Required Software**
 - If not already installed, download and install Python 3.5 or later from python.org.
 - Install SQL Server Express 2019 or later.
 - During installation, select "Named instance" and use the instance name "SQLEXPRESS".
2. **Set Up the Python Environment**
 - Open a command prompt or terminal.
 - Install the required Python libraries:
pip install PyQt5 pyodbc
3. **Get the RecipeJoy Application**
 - Clone the RecipeJoy repository:
git clone https://github.com/joywill24/RecipeJoy.git
4. **Set Up the Database**
 - Download the *RecipeJoy.bak* file from the Assets folder in the github repository.
 - Open SQL Server Management Studio (SSMS) and connect to your local SQLEXPRESS instance.
 - Right-click on "Databases" in the Object Explorer, select "Restore Database...".
 - Choose "Device" as the source, click the "..." button, and select "File" as the backup media.
 - Click "Add" and browse to the location of your downloaded RecipeJoy.bak file.
 - Click "OK" to start the restore process.
5. **Run the Application**
 - Open a command prompt or terminal.
 - Navigate to the RecipeJoy project directory.
 - Run the application:
python main.py

APPENDIX D (DEVELOPER SETUP INSTRUCTIONS)

The following provides instructions for setting up a development environment for RecipeJoy. Developers can follow these steps to prepare the system for working on the RecipeJoy codebase.

Prerequisites

Ensure the following are installed on your system:

- Python 3.5 or later
- Git
- SQL Server Express 2019 or later
- A code editor (ex, PyCharm)

Setup

1. Clone the repository
2. Install dependencies (PyQt5, pyodbc)
3. Download the RecipeJoy.bak database backup file from the GitHub repository
4. Restore the database
5. Set up IDE/Editor
6. Run the application
7. Set up testing environment

Development Workflow

1. Create a new branch for each feature or bug fix
2. Make your changes and commit regularly
3. Push your changes and create a pull request when ready for review
4. After approval, merge your changes into the main branch